Academic Project Review 3

Hotel Reservation and Guest Services Management System

The working application and final demonstration

Name — Dandigam Ritheek
Course — AI-ML
Institution — Woxsen University
Academic Year — 2026

Architecture

How the application is built

Operator

Types a menu choice at the terminal. Input is checked before it goes anywhere near the database.

→

main.py

Menu loop and 11 operations. Validates input, and catches every database error so it shows as one clear line instead of a crash.

→

db.py and reports.py

One connection helper plus an all-or-nothing transaction helper, with credentials read from a private .env file. Eight report wrappers. Values are always passed as query parameters.

→

MySQL 8 · hotel_pbl

12 tables, 8 views, 10 triggers, 14 foreign keys. The database enforces the business rules itself.

Python 3 · mysql-connector-python · python-dotenv · 585 lines of Python, 580 lines of SQL

Scope

Every required operation, mapped to a menu option

Required operation	Menu	Tables involved
Availability search	1	room, room_type, rate_plan, reservation
Reservation (create, cancel)	2, 10	guest, reservation
Check-in and check-out	3, 4	stay, reservation, room, invoice
Service posting	5	service_request, service_charge
Housekeeping assignment	6	housekeeping_task, staff, room
Billing	4, 8	invoice, service_charge, stay
Payment	7	payment, invoice
Reporting	9	8 views over the tables above
Guest management	11	guest

CRUD

Create, read, update and delete across the system

Record	Create	Read	Update	Delete
Guest	New guest while booking	Guest list, history report	Contact details (menu 11)	Menu 11, blocked if reservations exist
Reservation	Menu 2	Search, reports	Status changes on check-in	Cancel (menu 10) keeps the row for reports
Stay	Check-in	Occupancy report	Check-out time	Not offered, it is a record
Service request and charge	Menu 5	Service revenue report	Request marked fulfilled	Not offered, it is a record
Housekeeping task	Menu 6	Status report	Pending, in progress, done	Not offered
Invoice	At check-out	Menu 8	Status set automatically	Not offered, billing record
Payment	Menu 7	Invoice view	Not offered	Not offered, audit trail

Stay, invoice and payment records are deliberately never deleted: they are the audit trail. Rooms, rates and staff are maintained with SQL scripts.

Demonstration

One complete guest journey, run for real

1. Search 20 to 22 October: free rooms and rates are listed
2. Reserve room 202 (Deluxe) for Priya Sharma, 2 adults. Reservation 6 is created
3. Check in: stay 3 is created and the room becomes occupied
4. Log a Laundry request and post a 350.00 charge
5. Check out: invoice 2 is generated, total 4777.50
6. Record the 4777.50 payment by card: the invoice flips to paid
7. Assign a post-checkout cleaning task for room 202
8. Reports update straight away: occupancy, housekeeping and revenue

Search

Availability search: which rooms are free for these dates?

The query behind menu option 1 (dates are bound as parameters)

```
SELECT r.room_id, r.room_number, rt.type_name, rt.capacity, rp.nightly_rate
FROM room r
JOIN room_type rt ON r.type_id = rt.type_id
JOIN rate_plan rp ON rp.type_id = rt.type_id
WHERE r.status != 'maintenance'
  AND r.room_id NOT IN (
    SELECT room_id FROM reservation
    WHERE reservation_status NOT IN ('cancelled','no_show')
      AND %s < planned_check_out AND %s > planned_check_in)
```

Two date ranges overlap exactly when each starts before the other ends. The composite index on reservation(room_id, dates) serves this lookup.

Real output: 20 to 22 October 2026 (one row per room and rate plan)

room_number	type_name	capacity	nightly_rate
201	Deluxe	3	4200.00
202	Deluxe	3	4200.00
101	Standard	2	2500.00
101	Standard	2	2100.00
102	Standard	2	2500.00
102	Standard	2	2100.00
103	Standard	2	2500.00
103	Standard	2	2100.00
301	Suite	4	7500.00
301	Suite	4	8200.00

Billing

How the invoice is computed at check-out

- **Room charge** = nights stayed × the reservation's nightly rate. A same-day stay is billed as one night.
- **Services** = the sum of every service charge posted to the stay.
- **Tax** = 5% of the subtotal, rounded to 2 decimals.
- **Total** = subtotal + tax. Stored once, as a billing snapshot that never changes afterwards.
- **Payments** accumulate until the total is reached. The invoice moves open, then partially paid, then paid, set by a trigger. A payment that would exceed the total is rejected.

Real invoice from the demonstration

Item	Amount
Room charge (1 night × Deluxe Flexible)	4200.00
Services (Laundry)	350.00
Subtotal	4550.00
Tax (5%)	227.50
Total	**4777.50**
Invoice status after payment	paid

Validation

Three layers of validation

Layer	What it checks	Example from the demonstration
The application	Date format, whole numbers with a minimum, non-empty text, allowed choices, confirmation before cancel or delete	Invalid date format, try again (e.g. 2026-10-15).
Constraints in the database	8 CHECK, 5 UNIQUE, 14 foreign keys, NOT NULL, ENUM values	A guest cannot reuse another guest's email
Triggers in the database	Overlapping bookings, room capacity, payment limit, and three consistency rules between linked records	A booking that overlaps an existing one is refused

Because the rules live in the database, any client that connects (this app, Workbench, a future web app) gets the same protection.

Error handling

What the operator sees when something goes wrong

Action attempted	Message shown (captured from real sessions)
Book an overlapping date range	[Rejected] Room is already booked for an overlapping date range.
Type an impossible date	Invalid date format, try again (e.g. 2026-10-15).
Book 4 adults into a room for 2	[Rejected] Occupant count exceeds room type capacity.
Pay more than the invoice total	[Rejected] Payment would exceed the invoice total.
Reuse another guest's email	[Rejected] That value must be unique and already exists (e.g. an email already in use).
Delete a guest who has reservations	[Rejected] This record is still referenced by other data (e.g. the guest has reservations), so it cannot be deleted.
Cancel a reservation already checked in	Only 'booked' reservations can be cancelled (this one is 'checked_in').
Check out a stay that is already closed	Stay not found or already checked out.
Check in a cancelled reservation	Only 'booked' reservations can be checked in (this one is 'cancelled').

No crashes: every database error is caught and shown as one line. After the booking and payment attempts, the reservation and payment counts were unchanged.

Reports

Reports update the moment data changes — real output

Current occupancy

room_number	type_name	guest_name	actual_check_in
103	Standard	Arjun Mehta	2026-09-20 14:05:00

Total revenue by month

month	revenue_collected
2026-10	9187.50

Housekeeping status

task_id	room_number	task_type	assigned_staff	task_status
1	103	Daily cleaning	Ravi Kumar	done
3	302	Maintenance check	Lakshmi Devi	in_progress
2	101	Post-checkout cleaning	Ravi Kumar	pending
4	202	Post-checkout cleaning	Ravi Kumar	pending

Eight reports in total: occupancy, vacant rooms, cancellations, housekeeping status, guest history, service revenue, revenue by month and occupancy rate.

Testing

How the system was tested

Test	What was done	Result
Full workflow session	Search, reserve, check in, service charge, check out, pay, invoice, housekeeping, 7 reports, cancel, guest update and delete, all through the menu	Every step succeeded
Deliberate failures	Overlap, bad date, over capacity, overpayment, duplicate email, guest with reservations, cancelling a checked-in booking	All rejected with a one-line message
Business rules	Eight rules each broken directly in SQL	8 of 8 rejected, no data changed
Queries	13 labeled queries: joins, subqueries, aggregation, views, index check	All correct
Transaction test	The second step of a two-step transaction was made to fail	The first step was rolled back, nothing left behind
Crash check	Both sessions scanned for error traces	None found

Design choices

Security and design decisions

- **Dedicated database user.** The app connects as its own user with rights on hotel_pbl only, never as root.
- **Credentials stay out of the code.** They are read from a private .env file; only a blank .env.example is shared.
- **No SQL built from user text.** Every value the operator types is passed to MySQL as a parameter, which prevents SQL injection.
- **Rules live in the database.** Constraints and triggers protect the data whichever client connects.
- **All-or-nothing operations.** Check-in, check-out, service posting and finishing a housekeeping task each run as one transaction: if any step fails, everything is rolled back.
- **Soft cancel, hard history.** Cancelling keeps the reservation for reporting; stays, invoices and payments are never deleted.

Limitations

Limitations and future work

- Command-line interface for a single operator. The roles from Review 1 (receptionist, housekeeping, manager) are not enforced because there is no login. *Future: a web interface with role-based access.*
- Room types, rooms, rate plans and staff are maintained with SQL scripts, not through the menu.
- Tax is fixed at 5% and billing uses whole nights, with a minimum of one.
- Rate plans have no validity dates, so a plan applies all year.
- Two tables keep a documented, trigger-guarded exception to strict 3NF (see Review 2).

Submission

What is in the repository for Review 3

File	Purpose
app/main.py	Menu and 11 operations (461 lines)
app/db.py	Connection helper (87 lines)
app/reports.py	Report wrappers (37 lines)
db/01_schema.sql	Tables, keys, CHECK constraints
db/02_triggers.sql	10 triggers
db/03_sample_data.sql	Test data for every table
db/04_views_reports.sql	8 reporting views
db/05_queries.sql	13 labeled demonstration queries
docs/	Session transcript, validation tests, viva preparation

Thank You

Questions & Technical Discussion